

FedTransferAD: Federated Cross-System Microservice Anomaly Detection with Semantic Metric Harmonization

Abstract—Operational telemetry is rarely identical across microservice applications. Metric names, service counts, and normal operating ranges change from one deployment to another, which makes a detector trained on one system difficult to reuse elsewhere. We study whether a small, semantically aligned metric representation can support federated transfer to an application that never participates in model training. FedTransferAD maps recognized raw metrics to seven semantic families and summarizes robust deviations as a shared 21-dimensional feature vector. Two source applications train a 22-parameter logistic classifier with sample-weighted Federated Averaging, while the third application is held out. For each trace, the first 25% is used only as a healthy reference for local median/MAD normalization and is excluded from both training and evaluation. The target provides no labels, model updates, threshold tuning, or hyperparameter selection. On the metric-only RE1 suite of RCAEval, leave-one-system-out evaluation across Online Boutique, Sock Shop, and Train Ticket gives mean F1 0.8905, ROC-AUC 0.9413, and average precision 0.9601 over three SGD seeds. Centralized pooling gives F1 0.9009, ROC-AUC 0.9395, and AP 0.9511. Thus the federated model remains close to centralized training in F1 and slightly exceeds it on the two ranking metrics. Linear SHAP applied to the exact saved checkpoints shows that CPU, latency, and memory account for most of the model attribution. The results suggest that a compact semantic metric interface can support data-local cross-system transfer, although the current study still depends on a healthy target burn-in and rule-based metric mapping.

Index Terms—federated learning, microservices, anomaly detection, cross-system transfer, semantic metric harmonization, explainable AI

I. INTRODUCTION

Microservice applications are observed through large collections of metrics, logs, and distributed traces. These signals make failures visible, but they also create a portability problem: the telemetry available in one application rarely has the same names, dimensions, or operating ranges as another. A CPU-related metric may be exposed under different service prefixes, the number of latency metrics may change with the topology, and a value that is normal in one application can be unusual in another. A detector that learns directly from raw telemetry can therefore become tightly coupled to the system on which it was built.

Recent work has improved anomaly detection by using richer application representations. ServiceAnomaly combines distributed traces with profiling metrics [1]; DiagFusion fuses traces, logs, metrics, and dependency information for diagnosis [2]; UAC-AD and MAD-CMC learn from multiple telemetry modalities with contrastive objectives [3], [4]. These

approaches show that detailed system context is useful, especially for diagnosis and root-cause analysis. They do not, however, remove the practical difficulty of moving a detector between applications whose raw telemetry schemas differ.

A second difficulty appears when several environments could benefit from collaborative training but raw operational data should remain local. Federated learning (FL) addresses this setting by exchanging model updates rather than training records [5]. FL has already been used for microservice anomaly detection [6], network-device monitoring [7], intrusion detection [8]. In most of these studies, the evaluated devices or services are themselves members of the federation. Our question is different: can two source applications train a shared detector that transfers to a third application that was absent from model training?

FedTransferAD addresses this question with a deliberately lightweight design. Each application converts recognized metrics into the same seven semantic families—CPU, memory, latency, error, load, disk I/O, and socket activity—and summarizes robust deviations within each family. The resulting 21 features are independent of the number of mapped raw metrics. Two source applications then train a logistic classifier through FedAvg, while the third application is held out. The target is not fully data-free: it uses a short known-normal burn-in to estimate its own median and MAD, but it does not contribute labels or classifier updates.

The study makes four contributions. First, it defines a leave-one-system-out federated transfer setting in which the target application is excluded from model learning. Second, it introduces a fixed semantic metric interface for heterogeneous raw schemas. Third, it evaluates all three source–target combinations of RCAEval RE1 using a non-overlapping burn-in protocol and matched centralized and single-source baselines. Finally, it explains the exact trained checkpoints with Linear SHAP and reports the communication cost of the 22-parameter global model.

II. RELATED WORK

A. Microservice anomaly detection and cross-system transfer

Microservice anomaly detectors increasingly combine multiple forms of telemetry. Panahandeh et al. model distributed traces together with profiling metrics and report F1 scores up to 0.86 on TeaStore and TrainTicket [1]. Zhang et al. combine traces, logs, metrics, and dependency information to localize failures and classify their type [2]. Liu et al. use

adversarial contrastive learning over aligned logs and metrics [3], while Wang et al. model metric and log representations with a cross-modal Transformer and contrastive clustering [4]. These methods provide a broad view of system behavior, but their representations are built around the telemetry available in the evaluated systems.

Cross-system work is less common. LogGT transfers anomaly knowledge between log systems using heterogeneous graphs, semantic weighting, and domain adaptation, with reported cross-system F1 scores from 0.644 to 0.978 [9]. Its setting is useful evidence that transfer between software systems is feasible, but it is log-specific and adapts using target-domain information. FedTransferAD studies a narrower metric-only setting and keeps the classifier fixed at the target; only local normalization uses target observations.

B. Federated anomaly detection

Hao et al. are the closest microservice FL study in the reviewed literature. Their multi-task framework detects and classifies anomalies using metric time series from Sock-Shop, Train-Ticket, and several public time-series datasets [6]. The clients being evaluated participate in the federated learning problem. FedTransferAD instead holds out the entire target application from model training. Outside microservices, OmniFed clusters heterogeneous network devices and trains cluster-specific autoencoder models with FL [7]. Representative intrusion-detection work also explores decentralized and differentially private FL [8]. These studies establish that federated anomaly learning is practical, but they do not directly address transfer to a non-participating microservice application with a different raw metric schema.

C. Explainability and research gap

Explainability in microservice research is often tied to diagnosis rather than binary anomaly detection. DeepHunt uses multimodal service graphs and an interpretable reconstruction/propagation score to localize failures [10], while DiagFusion provides root-cause and failure-type predictions from multimodal telemetry [2]. FedTransferAD makes a more modest claim: SHAP is used to show which semantic metric families influence the classifier, not to identify a causal root service. The gap addressed here is therefore the combination of (i) source-only federated training, (ii) an entire microservice application held out from learning, (iii) a common representation for heterogeneous metric schemas, and (iv) lightweight semantic attribution.

III. FEDTRANSFERAD

A. Design overview and learning boundary

Let $\mathcal{S} = \{S_1, \dots, S_K\}$ be the source applications and let T be the held-out target. Source S_k owns labeled observations $D_k = \{(x_i, y_i)\}$ after burn-in removal. Raw observations never leave the source. Instead, every application implements a local mapping $\phi_k(\cdot) \in \mathbb{R}^{21}$ and the coordinator receives only model parameters. The important boundary is shown in Fig. 1: the target receives the final classifier, but it never sends a model update back to the federation.

B. Semantic metric harmonization

For each raw metric $x_j(t)$, the first 25% of the trace defines a healthy burn-in set B_j . Its robust location and scale are

$$m_j = \text{median}(B_j), \quad (1)$$

$$s_j = \max(1.4826 \text{MAD}(B_j), 10^{-6}). \quad (2)$$

Each value is converted to a clipped robust deviation,

$$z_j(t) = \min\left(50, \frac{|x_j(t) - m_j|}{s_j}\right). \quad (3)$$

Recognized metrics are then grouped into seven semantic families. For family f , three statistics are retained:

$$\mu_f(t) = \frac{1}{|f|} \sum_{j \in f} z_j(t), \quad (4)$$

$$M_f(t) = \max_{j \in f} z_j(t), \quad (5)$$

$$D_f(t) = \frac{1}{|f|} \sum_{j \in f} \mathbf{1}[z_j(t) > 3]. \quad (6)$$

The final representation $\phi(x_t) = [\mu_f, M_f, D_f]_{f=1}^7$ therefore has 21 dimensions regardless of how many recognized raw metrics are present. Missing families are zero-filled, and derived features are divided by a fixed factor of 10 before optimization. The mapper is intentionally simple rather than ontology-free: a new naming convention may require an additional mapping rule. In RE1, CPU, memory, latency, error, and load are active; disk-I/O and socket columns are absent and therefore produce zero-valued features.

C. Federated classifier and held-out inference

Each source client trains a logistic classifier,

$$p(y = 1 | x) = \sigma(w^\top \phi(x) / 10 + b). \quad (7)$$

At communication round r , source k starts from $\theta_r = (w_r, b_r)$, performs E local SGD epochs, and returns $\theta_r^{(k)}$ and its sample count n_k . The server computes

$$\theta_{r+1} = \sum_{k=1}^K \frac{n_k}{\sum_i n_i} \theta_r^{(k)}. \quad (8)$$

The model contains only 22 scalars. With 64-bit values, one upload or download contains 176 bytes; for two clients and 30 rounds, the analytical model payload is 20.6 KiB, excluding protocol and serialization overhead.

After training, the fixed global model is sent to the held-out application. The target constructs the same 21 features from its own local reference window and predicts the retained post-burn-in observations at threshold 0.5. Target labels are used only afterward to compute offline evaluation metrics. Linear SHAP [11] is applied to the exact saved checkpoints and aggregated by semantic family. The resulting values explain the model's decision surface, not the physical cause of a failure.

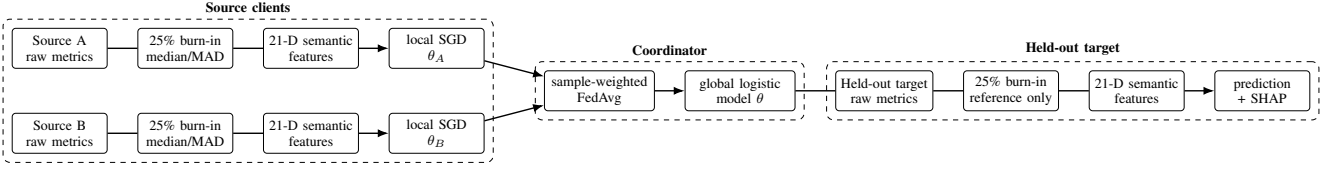


Fig. 1. FedTransferAD separates model learning from target-side preprocessing. Source applications exchange only model parameters. The target uses a local healthy reference window to normalize its metrics and to define the SHAP background, but it does not participate in classifier training.

Algorithm 1. FedTransferAD training and transfer

```

1: Input: sources  $\mathcal{S}$ , target  $T$ , burn-in  $q = 0.25$ , rounds  $R$ ,
   local epochs  $E$ 
2: for each source and target trace do
3:   estimate median/MAD from the first  $q$  fraction
4:   build 21-D semantic features and discard burn-in rows
5: end for
6: initialize global parameters  $\theta_0$ 
7: for  $r = 0, \dots, R - 1$  do
8:   for each source  $S_k$  do
9:     update  $\theta_r$  locally for  $E$  epochs
10:  end for
11:  aggregate source models with sample-weighted FedAvg
12: end for
13: send  $\theta_R$  to  $T$ ; predict retained target rows
14: explain  $\theta_R$  with Linear SHAP using target burn-in as reference

```

IV. EXPERIMENTAL SETUP

A. Dataset and non-overlapping protocol

We use the metric-only RE1 suite of RCAEval [12]. It contains 375 controlled fault-injection cases across Online Boutique (OB), Sock Shop (SS), and Train Ticket (TT), with 125 cases per application. Each application covers five fault types—CPU, memory, disk I/O, network delay, and packet loss—across five services with five repetitions per service–fault pair.

For each leave-one-system-out experiment, runs 1–3 of the two source applications provide labeled training data, while runs 4–5 of the held-out application form the target evaluation set. RCAEval provides the fault-injection timestamp for every case. Timestamps before injection are labeled normal; timestamps at or after injection are labeled anomalous. Because injection occurs near the midpoint, the original traces contain approximately equal healthy and faulty intervals.

The primary protocol avoids reuse of the normalization window. The first 25% of every trace is used only to estimate median/MAD statistics and is then removed. Source training and target scoring therefore begin after the burn-in. The target still contains a held-out healthy interval between 25% and the injection point, followed by the anomalous interval. Table I gives the resulting timestamp counts.

B. Baselines and implementation settings

For every held-out target we compare three training strategies: (i) a logistic model trained on each source separately, (ii) centralized logistic regression on the pooled source records, and (iii) FedTransferAD. All methods use the same 21 features, source labels, threshold 0.5, learning rate 0.3, and batch size 256. FedTransferAD runs for 30 communication rounds

TABLE I
DATA USED AFTER THE 25% BURN-IN IS REMOVED.

Sources $\rightarrow$ Target	Source train	Target eval	Anomalies
OB+SS $\rightarrow$ TT	156,207	34,250	22,850
OB+TT $\rightarrow$ SS	167,062	27,050	18,050
SS+TT $\rightarrow$ OB	91,725	78,109	51,328

TABLE II
FEDTRANSFERAD PERFORMANCE ON EACH HELD-OUT TARGET (MEAN $\pm$ SAMPLE SD OVER THREE SEEDS).

Target	F1	ROC-AUC	AP
OB	0.8438 $\pm$ 0.0066	0.9430 $\pm$ 0.0012	0.9531 $\pm$ 0.0007
SS	0.9481 $\pm$ 0.0003	0.9563 $\pm$ 0.0000	0.9723 $\pm$ 0.0000
TT	0.8795 $\pm$ 0.0037	0.9245 $\pm$ 0.0003	0.9549 $\pm$ 0.0002
Mean	0.8905	0.9413	0.9601

with five local epochs per source client. Centralized and single-source baselines run for 150 epochs, matching the source-local epoch budget. We repeat the experiments for SGD seeds 0, 1, and 2. These seeds measure optimization variability on the same benchmark split; they are not independent dataset replications.

We report F1, ROC-AUC, and average precision (AP). F1 captures performance at the fixed threshold, while ROC-AUC and AP summarize ranking behavior across thresholds. AP remains sensitive to class prevalence, which is important here because removing the burn-in leaves more anomalous than normal target timestamps.

V. RESULTS

A. Held-out-system detection

The first question is whether a model trained only on the two source applications remains useful on the third. Table II shows that it does, although the difficulty varies by target. SS is the easiest transfer case, with F1 0.9481, whereas OB is the hardest at 0.8438. TT lies between them at 0.8795. Across the three held-out systems, the mean is 0.8905 F1, 0.9413 ROC-AUC, and 0.9601 AP.

The spread across targets is informative. A single overall mean would hide that the transfer problem is not equally difficult in all directions. In particular, OB remains challenging despite a strong ROC-AUC of 0.9430, suggesting that ranking quality is better than the fixed-threshold F1 alone implies.

TABLE III
MEAN PERFORMANCE ACROSS THE THREE HELD-OUT TARGETS.

Method	F1	ROC-AUC	AP
Single-source (mean)	0.8911	0.9325	0.9540
Centralized pooled	0.9009	0.9395	0.9511
FedTransferAD	0.8905	0.9413	0.9601

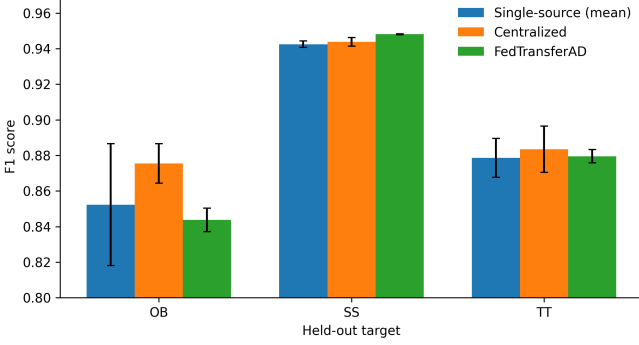


Fig. 2. Target-wise F1 comparison. The figure complements Table III by showing that the relative behavior of federation changes with the held-out application. Error bars are sample SD; the single-source bar also reflects the choice of source application.

B. Federated versus centralized and single-source learning

The second question is what is lost, or gained, by keeping the source datasets separate. Table III summarizes the three training strategies. Centralized pooling gives the highest mean F1 (0.9009), ahead of FedTransferAD by 0.0104. The federated model, however, gives slightly higher mean ROC-AUC (+0.0018) and AP (+0.0090). The mean single-source F1 is 0.8911, almost identical to the federated mean, but that average again hides target-specific behavior.

For that reason, Fig. 2 shows the F1 values separately for OB, SS, and TT. The plot should be read as a transfer profile rather than a leaderboard: federation is strongest relative to the alternatives on SS, remains close to centralized training on TT, and shows its largest F1 gap on OB. It also does not beat the best individual source model on every target. The practical value of federation in this experiment is therefore collaborative, data-local training with near-centralized utility, not guaranteed accuracy improvement in every transfer direction.

C. Semantic attribution

The third question is whether the transferred classifier relies on interpretable system-level signals rather than arbitrary raw metric identifiers. We therefore aggregate absolute SHAP values by semantic family. Figure 3 shows the mean attribution across the three target scenarios. CPU, latency, and memory dominate, while load and error contribute much less. This is consistent with the model relying mainly on broad resource and response-time deviations after raw schemas have been mapped into the common representation.

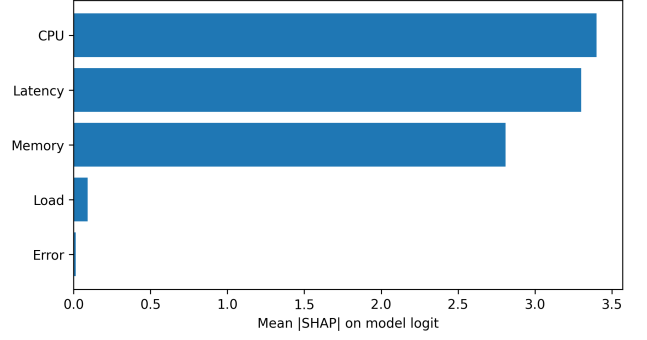


Fig. 3. Average semantic-family attribution of the saved FedTransferAD checkpoints. CPU, latency, and memory account for most of the model’s absolute SHAP magnitude. Values are measured on the logistic-model logit.

The ordering is not identical for every target. CPU is strongest for TT and SS, whereas latency is strongest for OB. Across the five non-degenerate families, pairwise Spearman correlations are 0.90 (TT–SS), 0.70 (TT–OB), and 0.60 (SS–OB). At the 15-feature level, the corresponding correlations are 0.975, 0.911, and 0.930. These are descriptive consistency measures rather than independent significance tests. Disk-I/O and socket attributions are zero because RE1 has no corresponding raw columns; they should not be interpreted as universally unimportant signals.

VI. DISCUSSION

A. How the results relate to previous work

Direct numerical comparison with earlier papers would be misleading because the tasks, modalities, datasets, and evaluation protocols are different. A more useful comparison is to ask what each line of work demonstrates. Hao et al. show that federated learning can support anomaly detection and classification inside a microservice federation [6]. In their setting the evaluated clients belong to the federated problem. FedTransferAD adds a different stress test: the target application is withheld from all classifier training. The roughly one-point F1 gap to centralized pooling can therefore be viewed as the cost of combining data-local source learning with cross-application transfer under the present representation.

LogGT is closer from the transfer perspective. It demonstrates cross-system anomaly detection for logs and uses target-domain adaptation to align heterogeneous graph representations [9]. FedTransferAD uses a more restrictive target model boundary: the target provides a healthy normalization reference, but the classifier weights and threshold are not adapted. The two studies therefore address complementary forms of portability—LogGT through learned log-domain adaptation and FedTransferAD through a fixed semantic metric interface.

The multimodal microservice literature addresses a different trade-off. ServiceAnomaly, DiagFusion, UAC-AD, MAD-CMC, and DeepHunt exploit traces and/or logs in addition to metrics to capture richer system context [1]–[4], [10]. This can support stronger diagnosis and root-cause reasoning, but it also

assumes the required telemetry is collected and represented consistently. FedTransferAD deliberately starts with metrics because they provide a compact common surface for testing federated transfer. Its SHAP output is correspondingly lighter: it identifies influential semantic families but does not localize a faulty service or causal path.

B. Practical interpretation

The main result is not that federated learning universally improves accuracy. It does not. Instead, the experiment shows that a 22-parameter model trained from two separate source systems can remain close to a centrally pooled model when moved to a third system with a different raw metric schema. The communication footprint is small, and the target requires no labeled adaptation. This makes the approach attractive as a lightweight baseline for environments where raw telemetry cannot be pooled but a short healthy commissioning period is available locally.

The target-specific differences also matter. OB is consistently harder than SS, which suggests that source selection and domain similarity may be as important as the aggregation rule itself. A natural extension is therefore to weight source clients according to their relevance to a new target rather than only by sample count.

C. Limitations and future work

The current study has several limitations. First, the evaluation is restricted to metric telemetry. Although metrics are lightweight and widely available, logs and traces can provide complementary information such as event semantics, error messages, request paths, and service dependencies. Extending the framework to multimodal telemetry could therefore improve both anomaly detection and diagnostic capability.

Second, the method assumes the availability of a healthy target burn-in period for normalization. This is reasonable in a controlled setting, but it may be harder to guarantee in real deployments where the observed data can contain noise, concept drift, or previously unseen faults. Future work should investigate more robust normalization strategies that can tolerate imperfect or changing target conditions.

Third, the experiments use only three microservice applications with controlled fault injections. While the leave-one-system-out protocol provides a meaningful cross-system evaluation, a larger number of applications and more diverse failure scenarios would provide stronger evidence of generalizability. Validation on additional benchmarks and real production telemetry would be especially useful.

Finally, the current approach relies on rule-based metric-family mapping and standard FedAvg. These choices keep the framework simple and interpretable, but they do not provide formal privacy guarantees or root-cause localization. Future extensions could explore learned semantic mappings, stronger privacy mechanisms such as secure aggregation or differential privacy, and service-level diagnostic methods for identifying the source of failures.

VII. CONCLUSION

FedTransferAD examines a specific but practical question: whether federated source learning can transfer an anomaly detector to a microservice application that never participates in model training. A fixed 21-dimensional semantic metric representation makes the classifier independent of the number of recognized raw metrics, while a non-overlapping 25% healthy burn-in gives each application its own robust normalization reference. On RCAEval RE1, the federated model achieves mean F1 0.8905, ROC-AUC 0.9413, and AP 0.9601 across the three held-out targets. Centralized training remains slightly better in F1, but the gap is small and the federated model is competitive on the ranking metrics. SHAP analysis shows that CPU, latency, and memory dominate the transferred predictions. These findings position FedTransferAD as a lightweight baseline for data-local cross-system monitoring rather than a complete diagnostic system. Extending the same transfer principle to multimodal telemetry, more systems, robust target commissioning, and stronger privacy protection is the next step.

REFERENCES

- [1] M. Panahandeh, A. Hamou-Lhadj, M. Hamdaqa, and J. Miller, "ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *J. Syst. Softw.*, vol. 209, Art. no. 111917, 2024.
- [2] S. Zhang *et al.*, "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 3851–3864, 2023.
- [3] H. Liu *et al.*, "UAC-AD: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3887–3900, 2024.
- [4] P. Wang, X. Zhang, Y. Chen, and Z. Cao, "Unsupervised microservice system anomaly detection via contrastive multi-modal representation clustering," *Inf. Process. Manag.*, vol. 62, Art. no. 104013, 2025.
- [5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [6] J. Hao, P. Chen, J. Chen, and X. Li, "Multi-task federated learning-based system anomaly detection and multi-classification for microservices architecture," *Future Gener. Comput. Syst.*, vol. 159, pp. 77–90, 2024.
- [7] S. Zhang *et al.*, "Privacy-preserving MTS anomaly detection for network devices through federated learning," *Inf. Sci.*, vol. 690, Art. no. 121590, 2025.
- [8] O. Friha *et al.*, "2DF-IDS: Decentralized and differentially private federated learning-based intrusion detection system for industrial IoT," *Comput. Secur.*, vol. 127, Art. no. 103097, 2023.
- [9] P. Wang, X. Zhang, Z. Cao, W. Xu, and W. Li, "LogGT: Cross-system log anomaly detection via heterogeneous graph feature and transfer learning," *Expert Syst. Appl.*, vol. 251, Art. no. 124082, 2024.
- [10] Y. Sun *et al.*, "Interpretable failure localization for microservice systems based on graph autoencoder," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Art. no. 52, 2025.
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4765–4774.
- [12] L. Pham *et al.*, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proc. ACM Web Conf.*, 2025, pp. 777–780.